

Reflexión Evidencia 1

César Cárdenas – A00844803

Importancia de ordenar y buscar logs

En esta evidencia trabajé con dos archivos de logs. Cada registro tenía una fecha, hora, dirección IP y un mensaje. Uno de los archivos estaba desordenado y el otro estaba casi ordenado, aunque tenía algunos registros repetidos.

Ordenar los logs es importante porque así podemos saber en qué orden pasaron los eventos. Esto puede servir para revisar un error del sistema o algún problema de seguridad. Si los registros están desordenados, sería más difícil entender qué ocurrió primero.

Después de ordenar también se puede hacer una búsqueda por fechas. En este caso utilicé búsqueda binaria, ya que es más rápida que revisar los 6,818 registros uno por uno. Su complejidad es $O(\log n)$, pero solo funciona correctamente cuando los datos ya están ordenados.

Comparación de algoritmos

En el programa implementé Swap Sort, Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort y Shell Sort. Shell Sort fue el algoritmo extra que agregué en la Actividad 1.5.

Swap Sort compara cada registro con todos los siguientes. Es sencillo, pero puede hacer demasiadas comparaciones e intercambios. Su complejidad es $O(n^2)$.

Selection Sort busca el registro menor y lo coloca en su posición. También es $O(n^2)$ y no cambia mucho si el archivo está casi ordenado, porque de todas formas sigue buscando el menor en el resto del vector.

Bubble Sort compara registros vecinos. Mi versión completa todas las vueltas aunque los datos ya estén ordenados. Por eso su mejor y peor caso son $O(n^2)$. Podría mejorar si terminara cuando ya no existen intercambios.

Insertion Sort va colocando cada registro en la parte que ya está ordenada. Su mejor caso es $O(n)$ y el peor es $O(n^2)$. Este algoritmo puede funcionar bien con log607-2.txt porque muchos registros ya están cerca de su posición.

Merge Sort divide los datos y después vuelve a unirlos en orden. Su complejidad es $O(n \log n)$ en el mejor y peor caso. Me pareció de los algoritmos más estables, aunque usa un vector temporal.

Quick Sort utiliza un pivote para separar los datos. Normalmente es $O(n \log n)$, pero puede llegar a $O(n^2)$. En mi programa se toma el último registro como pivote, por lo que podría funcionar peor si los datos ya están muy ordenados.

Shell Sort utiliza espacios entre los elementos y va reduciendo esos espacios. En mis pruebas fue rápido con el archivo casi ordenado, ya que no tuvo que mover demasiados registros.

Resultados y predicciones

Los dos archivos tenían 6,818 registros. Antes de cada corrida escribí una predicción sobre si esperaba que el algoritmo fuera rápido o lento.

Selection Sort tardó 802,415 microsegundos con log607-1.txt. Merge Sort tardó 75,323 microsegundos con el mismo archivo. La diferencia fue grande y coincide con la teoría, porque Selection Sort es $O(n^2)$ y Merge Sort es $O(n \log n)$.

También probé Shell Sort varias veces con log607-2.txt. Los tiempos fueron 14,661, 25,902 y 24,769 microsegundos. Mi predicción era que sería rápido porque el archivo estaba casi ordenado. En general sí coincidió con lo que esperaba.

Los tiempos no salieron exactamente iguales en todas las pruebas. Supongo que esto pasó porque la computadora también estaba ejecutando otras cosas. Aun así, se pudo observar que los algoritmos no se comportan igual con datos desordenados y casi ordenados.

Por ejemplo, Insertion Sort puede aprovechar el orden previo, pero Selection Sort sigue realizando casi las mismas comparaciones. Merge Sort tiene un comportamiento más parecido en ambos archivos porque siempre divide y combina los datos.

Búsqueda por rango y duplicados

Para la búsqueda por rango utilicé dos búsquedas binarias. buscarInicio encuentra el primer registro igual o mayor que la fecha inicial. buscarFin encuentra la primera posición después de la fecha final.

Fue necesario hacerlo así porque log607-2.txt contiene timestamps duplicados. Una búsqueda binaria normal puede encontrar uno de los registros repetidos, pero no necesariamente el primero.

Con las dos búsquedas se incluyen todos los registros duplicados sin repetirlos ni perderlos. Al final, los resultados encontrados se guardan en range607.txt.

Uso de inteligencia artificial

Utilicé inteligencia artificial para resolver dudas de sintaxis, revisar errores y adaptar los algoritmos que ya tenía en la Actividad 1.5.

Algunos prompts que utilicé fueron:

1. “¿Cómo puedo comparar registros Log usando su fecha y hora?”
2. “¿Cómo adapto los algoritmos de vector<int> a vector<Log>?”

3. “¿Cómo hago una búsqueda binaria que incluya timestamps duplicados?”

No todas las sugerencias funcionaron correctamente desde el inicio. Uno de los errores estuvo en la creación de la clave. Se había sugerido juntar directamente el año, el nombre del mes, el día y la hora.

Eso no funcionaba bien porque los meses se comparaban alfabéticamente. También existía un problema porque el día 2 no tenía la misma cantidad de caracteres que el día 10.

Lo corregí convirtiendo los meses a números, por ejemplo Jan a 01 y Sep a 09. También agregué un cero a los días menores que 10. Así la clave quedó formada por año, mes, día y hora.

Otro error fue la lectura del archivo. Al principio el año se estaba guardando como si fuera la hora y los demás datos quedaban recorridos. Me di cuenta cuando ejecuté el programa y vi que la salida no tenía sentido. Después corregí la lectura para separar month, day, year, time e ip.

También tuve que revisar los límites de la búsqueda binaria. Buscar solamente una coincidencia podía dejar fuera algunos timestamps repetidos. Por eso terminé usando una búsqueda para el inicio y otra para el final.

Esto me ayudó a entender que no debo aceptar una respuesta de IA solo porque el código compila. También tengo que revisar las salidas y entender qué está haciendo.

Conclusión

Con esta actividad entendí mejor cómo cambia el tiempo entre algoritmos $O(n^2)$ y $O(n \log n)$. Los algoritmos sencillos son más fáciles de programar, pero pueden tardar bastante cuando hay muchos registros.

Merge Sort tuvo un resultado más estable y Shell Sort funcionó bien con el archivo casi ordenado. También entendí que el orden inicial de los datos puede ayudar mucho a algunos algoritmos, pero a otros no tanto.

La búsqueda binaria fue una parte importante porque tuve que manejar correctamente los límites y los registros duplicados. Finalmente, la inteligencia artificial me sirvió como apoyo, pero tuve que probar y corregir varias sugerencias antes de usarlas.